Flight Procedures Manual

Project Zephyrus

J. Bolte, M. Nichitiu, S. Parke

MIT Rocket Team Avionics

Last Updated: JAN 18 2026

MIT ROCKET TEAM

Instructions

This manual is for use on Project Zephyrus flights, to be printed for launch in a minimum of 2 copies, one for the Telemetry Lead, and another for the Telemetry Deputy. It is to be read prior to launch by all members involved in the Ground Station, Away Station, and Avionics/Structures Lead roles. At each of the Away Station(s) and Ground Station, 2 copies must be printed and be available for the site lead and deputy to access prior to, during, and after flight. The personnel whose mastery of this document's contents is listed below:

- **Avionics Lead**
  (Sections 1-9)
- **Ground Station Lead**
  (Sections 1-2, 4-6, 8)
- **Ground Station Deputy**
  (Sections 1-2, 4-6)
- **Away Station Lead**
  (Sections 3-6, 9)
- **Away Station Deputy**
  (Sections 3-6, 9)
- **Simulation Lead**
  (Sections 1-10)
- **Structures Lead**
  (Sections 1, 4-6)

Contents

1 Avionics Setup Instructions 2
1.1 Avionics Bay 2
1.2 Fin Control 2
1.3 Airbrakes Module 2
1.4 Inter-Subsystem Connections 2
2 Ground Station Setup Instructions . 3
2.1 Telemetry Reception 3
2.2 Live Video Reception 3
3 Away Station Setup Instructions . . 4
3.1 Antenna Pointer Assembly . . 4
3.2 Antenna Pointer Setup 4
4 Pre-Flight Procedures 5
4.1 Go-No Go Criteria 5
5 In-Flight Procedures 6
5.1 Expected Behavior 6
5.2 Flight Detection Failure Procedure 6
5.3 Flight Computer Reset Procedure 6
5.4 Lost Connection Procedure . . 6
6 Post-Flight Procedures 7
7 Onboard Electronics Documentation . 8
7.1 Onboard Hardware Documentation 8
7.2 Onboard Software Documentation 8
8 Ground Station Documentation . . . 9
8.1 Ground Station Hardware . . . 9
8.2 Ground Station Software . . . 9
9 Away Station Documentation 10
9.1 Away Station Hardware 10
9.2 Away Station Software 10
10 FC Simulation Documentation 11
10.1 Usage Instructions 11
10.2 Usage Examples 11
10.3 FC Simulation Framework . . . 11
10.4 OR Simulation Framework . . . 11
10.5 Python Scripting Framework . 11
11 Airbrakes Control Algorithm 12

1 Avionics Setup Instructions

1.1 Avionics Bay

1.2 Fin Control

1.3 Airbrakes Module

1.4 Inter-Subsystem Connections

2 Ground Station Setup Instructions

2.1 Telemetry Reception

2.2 Live Video Reception

3 Away Station Setup Instructions

3.1 Antenna Pointer Assembly

3.2 Antenna Pointer Setup

4 Pre-Flight Procedures

4.1 Go-No Go Criteria

5 In-Flight Procedures

5.1 Expected Behavior

5.2 Flight Detection Failure Procedure

TLDR advance state manually

5.3 Flight Computer Reset Procedure

TLDR assure visual apogee +1 second, arm and fire pyros manually; enter APOGEE state/`DISREEF_AWAIT` state manually

5.4 Lost Connection Procedure

TLDR cooked, antenna proc maybe

6 Post-Flight Procedures

7 Onboard Electronics Documentation

7.1 Onboard Hardware Documentation

7.2 Onboard Software Documentation

8 Ground Station Documentation

8.1 Ground Station Hardware

8.2 Ground Station Software

9 Away Station Documentation

9.1 Away Station Hardware

9.2 Away Station Software

10 FC Simulation Documentation

10.1 Usage Instructions

10.2 Usage Examples

10.3 FC Simulation Framework

10.4 OR Simulation Framework

10.5 Python Scripting Framework

11 Airbrakes Control Algorithm

Theory

Consider a rocket of mass m moving through air of density ρ. Let x be the vertical motion. We approximate the rocket to move in a purely vertical fashion, in order to simplify calculations. The differential equation governing the rocket's motion is

$$m\ddot{x} = -mg - \frac{\rho C_{D_{\text{rkt}}} A_{\text{ref}}}{2}\dot{x}^2 \tag{11.1}$$

We rearrange to obtain

$$\ddot{x} = -g - \xi\dot{x}^2 \;:\quad \xi \equiv \frac{\rho C_{D_{\text{rkt}}} A_{\text{ref}}}{2m} \tag{11.2}$$

Let $\Delta \equiv (t - t_{\text{apog}})$. This is a nonlinear differential equation without closed form, and so we seek to approximate its solution x as follows, knowing that $\ddot{x} = -g$ and $\dot{x} = 0$ at apogee:

$$\dot{x}(t) \approx a\Delta^3 + b\Delta^2 - g\Delta \tag{11.3}$$

so that

$$\ddot{x}(t) \approx 3a\Delta^2 + 2b\Delta - g \tag{11.4}$$

and

$$x(t) \approx \frac{a}{4}\Delta^4 + \frac{b}{3}\Delta^3 - \frac{g}{2}\Delta^2 + x_0 \tag{11.5}$$

where x_0 is the height at apogee.

From the figure at right we can deduce that this method is appropriate, assuming the t_{apog} apogee time can be properly estimated. For a linear least-squares fitter, the only kind feasible on the onboard flight computer, the t_{apog} must not be fitted at the same time as a and b; a simulation output cannot be used either since this would prevent flexibility in case of turbulence or ill-modeled motor performance. Thus, we use the simulator output ${}_0t_{\text{apog}}$ to generate three possibilities of apogee times, and fit a quintic to the acceleration of the following form to decide between the apogee times:

$$\tilde{a}(t) = a_0(t - t_{\text{apog}})^5 - g \tag{11.6}$$

The correlation coefficient R^2 is used to choose the best apogee time out of $[{}_0t_{\text{apog}} - 1,\ {}_0t_{\text{apog}},\ {}_0t_{\text{apog}} + 1]$. Then, a shift of `AIRBRAKES_T_APOG_FUDGEDIFF` is introduced, since the cubic approximation above functions best with an overestimated $\tilde{t}_{\text{apog}}$. Once the apogee time and altitude are correctly predicted, energy conservation is used to model the effect of the airbrakes on the apogee altitude[1]. The work done on the rocket by some airbrakes area function of time $A(t)$ is

$$W = \int F\ dx = \int \frac{C_D \rho A(t)\dot{x}^2}{2}\ dx \tag{11.7}$$

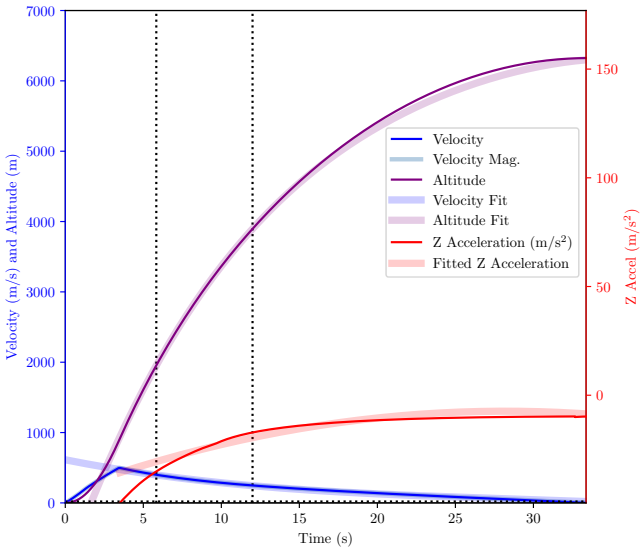

Figure 11.1. Fit $\tilde{\dot{x}}$ onto the blue velocity function of time simulator output. 13 points between the two vertical dotted lines were used to find the fit, for $t_{\text{apog}} = 35.5$ s. Red shows the derivative of the fit; purple shows the integral, with x_0 calculated from the right dotted line intersection with the purple curve. The difference between the predicted apogee altitude $\tilde{x}_0$ and the true apogee altitude x_0 is 3.5 m.

This will correspond to a change in altitude Δx in the rocket apogee height:

$$\int \frac{C_D \rho A(t)\dot{x}^2}{2}\ dx = mg(\Delta x) \tag{11.8}$$

A rapid change of variables using $\dot{x}dt = dx$ gives

$$\int \frac{C_D \rho A(t)\dot{x}^3}{2}\ dt = mg(\Delta x) \tag{11.9}$$

and so

$$\int A(t)\dot{x}^3\ dt = \frac{2mg(\Delta x)}{C_D \rho} \tag{11.10}$$

We are free to choose a model for $A(t)$ that is convenient to integrate and also physically actuate on the rocket. In this vein, we choose $A(t)$ to be a constant A_0 with a small ramp that lasts a half-second. As the integration will be quite tedious, we ignore the ramp in deriving the required airbrakes area, as that will be accounted for in the "fudge factor" that also accounts for lack of pure vertical motion, as well as other simplified phenomena.

[1] Note that we approximate the apogee time as remaining unchanged.

Thus, we solve

$$A_0 = \frac{2mg(\Delta x)}{C_D \rho} \left(\int_{t_0}^{t_{\text{apog}}} \left((a(t - t_{\text{apog}})^3 + b(t - t_{\text{apog}})^2 - g(t - t_{\text{apog}}) \right)^3 \, dt \right)^{-1} \tag{11.11}$$

The integral expression gives

$$\begin{aligned} \int_{t_0}^{t_1} \dot{x}^3 \, dt = -\Bigg(& \frac{a^3}{10} (t_0 - t_1)^{10} + \frac{a^2 b}{3} (t_0 - t_1)^9 + \frac{3ab^2 - 3a^2 g}{8} (t_0 - t_1)^8 \\ & + \frac{b^3 - 6abg}{7} (t_0 - t_1)^7 + \frac{ag^2 - b^2 g}{2} (t_0 - t_1)^6 + \frac{3bg^2}{5} (t_0 - t_1)^5 - \frac{g^3}{4} (t_0 - t_1)^4 \Bigg) \end{aligned} \tag{11.12}$$

and so if $T \equiv t_{\text{apog}} - t_0$,

$$A_0 = f \cdot \left(-\frac{2mg(\Delta x)}{C_D \rho} \cdot \frac{T^{-4}}{\frac{a^3}{10} T^6 + \frac{a^2 b}{3} T^5 + \frac{3ab^2 - 3a^2 g}{8} T^4 + \frac{b^3 - 6abg}{7} T^3 + \frac{ag^2 - b^2 g}{2} T^2 + \frac{3bg^2}{5} T - \frac{g^3}{4}} \right) \tag{11.13}$$

where f is the aforementioned "fudge factor," whose value is generally close to 3, tuned prior to launch. The latest values appear in the table below:

m (kg)	f for $\Delta x < 40$ m	f for $\Delta x \geq 40$ m
51.71	3.2	3.5

Table 1. "Fudge Factors" for Airbrakes Algorithm ⟨AS OF JAN 20 2026⟩

Onboard Algorithm

We have implemented the Airbrakes Control Algorithm using a state machine with the following states (encoded in `AirbrakesControllerState`):

- DISABLED
- PREP
- PREPROCESS
- WAIT_FOR_START
- CONTROLLING_RAMP
- CONTROLLING_PLATEAU
- DONE
- INFEASIBLE

Flow from one state to the next is governed according to the following diagram:

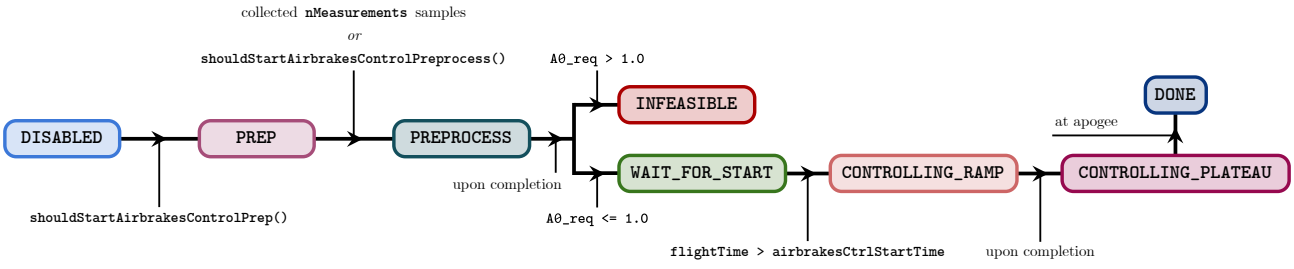

Figure 11.2. Airbrakes Controlled State Flow.

We detail the exact procedure.

- DISABLED

  The Airbrakes controller starts in this state. No action is taken until the method `shouldStartAirbrakesControlPrep()` returns `true`.

- PREP

  The PREP stage consists of sampling velocity $\dot{x}(t)$ and acceleration $\ddot{x}(t)$ a total of nMeasurements times, with timestamps saved as well. Should the START_AIRBRAKES_PREPROC_TIME marker have arrived prior to the full sample acquisition, the controller moves on to the PREPROCESS state.

- PREPROCESS

  The PREPROCESS stage consists of choosing the apogee time from the list of possibilities, according to the R^2 correlation coefficient of the quintic fit to acceleration (y_i points). The quintic fit is conducted by linear least squares, according to

$$\tilde{a} = \frac{\sum_i (y_i + g)(t_i - t_{\text{apog}})^5}{\sum_i (t_i - t_{\text{apog}})^{10}} \tag{11.14}$$

  and

$$R^2 = 1 - \frac{\sum_i y_i - f(x_i; \tilde{a})}{\sum_i (y_i - \overline{y})^2} \tag{11.15}$$

  The apogee time t_{apog} is decided to be $\tilde{t}_{\text{apog}}$ + AIRBRAKES_T_APOG_FUDGEDIFF. Next, the velocity is fitted to

$$\dot{x}(t) = a(t - t_{\text{apog}})^3 + b(t - t_{\text{apog}})^2 - g(t - t_{\text{apog}}) \tag{11.16}$$

  This is again performed with linear least squares. The $X^T X$ matrix is

$$X^T X = \begin{bmatrix} \sum_i (t_i - t_{\text{apog}})^6 & \sum_i (t_i - t_{\text{apog}})^5 \\ \sum_i (t_i - t_{\text{apog}})^5 & \sum_i (t_i - t_{\text{apog}})^4 \end{bmatrix} \tag{11.17}$$

  The $X^T \boldsymbol{y}$ matrix given by

$$X^T \boldsymbol{y} = \begin{bmatrix} \sum_i (t_i - t_{\text{apog}})^3 (\dot{x}_i + g(t_i - t_{\text{apog}})) \\ \sum_i (t_i - t_{\text{apog}})^2 (\dot{x}_i + g(t_i - t_{\text{apog}})) \end{bmatrix} \tag{11.18}$$

  The two parameters $\tilde{a}$ and $\tilde{b}$ are thus

$$\begin{bmatrix} \tilde{a} \\ \tilde{b} \end{bmatrix} = \left(X^T X\right)^{-1} X^T \boldsymbol{y} \tag{11.19}$$

  Next, the apogee altitude is obtained from

$$x_0 = x(t_0) - \frac{a}{4}(t_0 - t_{\text{apog}})^4 + \frac{b}{3}(t_0 - t_{\text{apog}})^3 - \frac{g}{2}(t_0 - t_{\text{apog}})^2 \tag{11.20}$$

  The desired Δx is obtained from the apogee prediction and the desired altitude, either obtained by rounding:

$$x_{\text{desired}} = \left\lfloor \frac{x_0}{n} \right\rfloor n \tag{11.21}$$

  where n is some integer detailing the up-to-the-closest quantity to round to. The airbrakes start time is

$$\texttt{airbrakesCtrlStartTime} = \texttt{currentTime} + \texttt{AIRBRAKES\_TIME\_DELAY} \tag{11.22}$$

  The airbrakes area required is

$$A_0 = \texttt{reqDeployedAreaAirbrakes}(\texttt{currentTime} + \texttt{AIRBRAKES\_TIME\_DELAY}, \Delta x) \tag{11.23}$$

  Should $A_0 > 1$, the state progresses to INFEASIBLE. Otherwise, the required A_0 is stored, and the state progresses to WAIT_FOR_START.

- WAIT_FOR_START

  Simply wait for the flight time to be past `airbrakesCtrlStartTime`. The state progresses to CONTROLLING_RAMP.

- CONTROLLING_RAMP

  The airbrakes area $A(t)$ here is given by

  $$A(t) = 2A_0(t - \texttt{airbrakesCtrlStartTime}) \tag{11.24}$$

  The state ends after 0.5 seconds, progressing to CONTROLLING_PLATEUAU.

- CONTROLLING_PLATEAU

  The airbrakes area $A(t)$ here is given by

  $$A(t) = A_0 \tag{11.25}$$

  The state ends at apogee, progressing to DONE.

- DONE

  Nothing occurs in this state.

- INFEASIBLE

  Nothing occurs in this state.

Values of Constants ⟨LAST UPDATED JAN 20⟩

```
double EARLIEST_START_AIRBRAKES_PREP_TIME = 4.0;
double START_AIRBRAKES_PREP_VEL           = 400.0;
double START_AIRBRAKES_PREPROC_TIME       = 12.0;
double AIRBRAKES_TIME_DELAY               = 1.0;
int    AIRBRAKES_N_MEASUREMENTS           = 13;
int    AIRBRAKES_MEASUREMENT_FREQ_HZ      = 5;
int    AIRBRAKES_SIMULATION_T_APOG        = 34;
double AIRBRAKES_T_APOG_FUDGEDIFF         = 1.5;
double fudge_factor                       = 3.2;
double fudge_factor_2                     = 3.5;
int    roundToHowMuch                     = 1000;
double AIRBRAKES_MAX_AREA                 = 0.0066;
```

Notes from Xanthus Launch

* Possibly detect flight from remote rocket without human assistance
    * FLIGHT -> FLIGHT, APOGEE will be detected asap anyway
    * PREFLIGHT -> PREFLIGHT, ensure FLIGHT detection would work from in-flight
    * MASTER PHYSICAL SWITCH: enable/disable set_state_from_SD_card on the rocket
    * GROUND STATION SWITCH: if enabled, allows for <something>